



NTU Academy for Professional
and Continuing Education

(SCTP) Advanced
Professional Certificate

Data Science and AI





NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

2.2 Data Architecture

Module Overview

2.1 Introduction to Big Data and Data Engineering

2.2 Data Architecture

2.3 Data Encoding and Data Flow

2.4 Data Extraction and Web Scraping

2.5 Data Warehouse

2.6 Data Pipelines

2.7 Testing and Data Orchestration

2.8 Out of Core/Memory Processing

2.9 Big Data Ecosystem and Batch Processing

2.10 Event Streaming and Stream Processing

Learning Objectives

Learners will be able to:

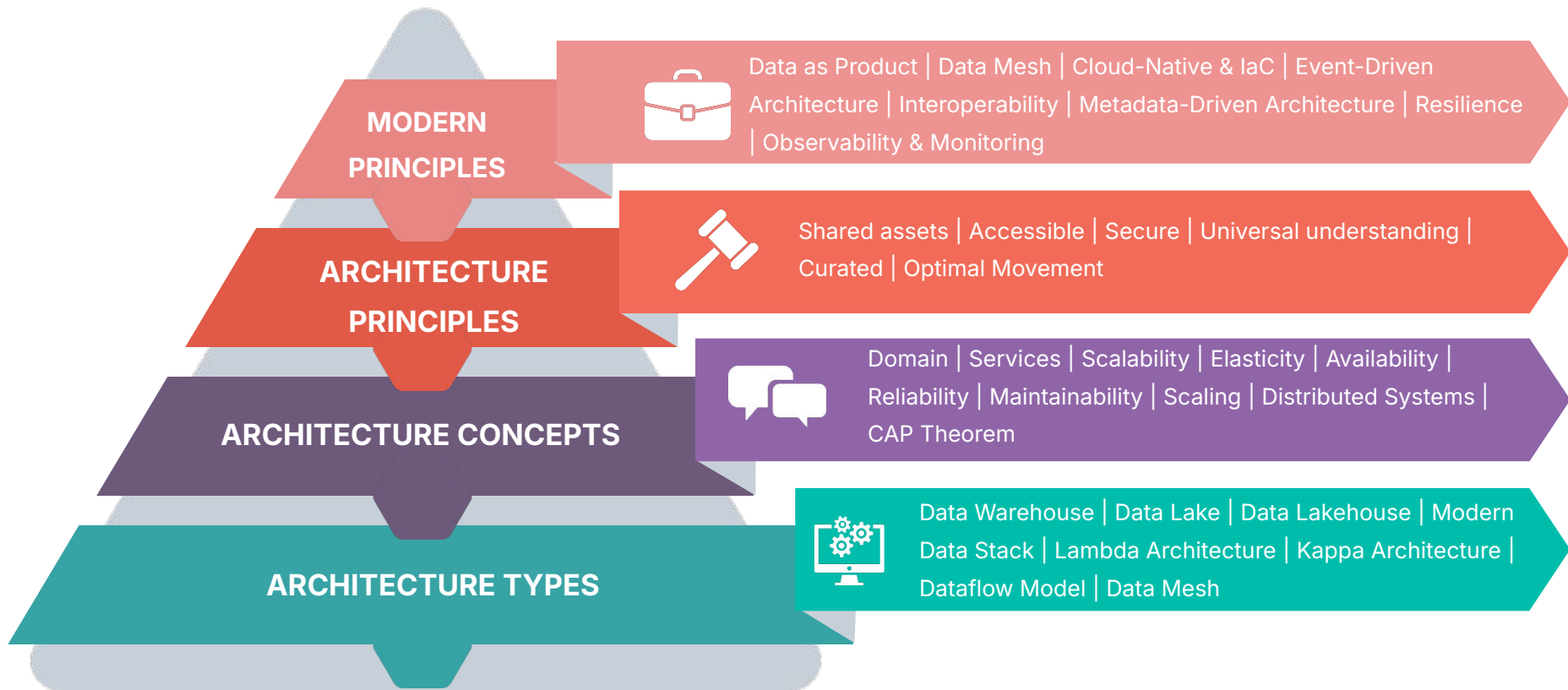
- Consider trade-offs between different data architecture types.
- Perform create and read operations on Redis.
- Fetch data from Google Cloud Storage.

2.1 Recap and 2.2 Pre-class Quiz

Join at
slido.com
#DS6202



AGENDA | Data Architecture



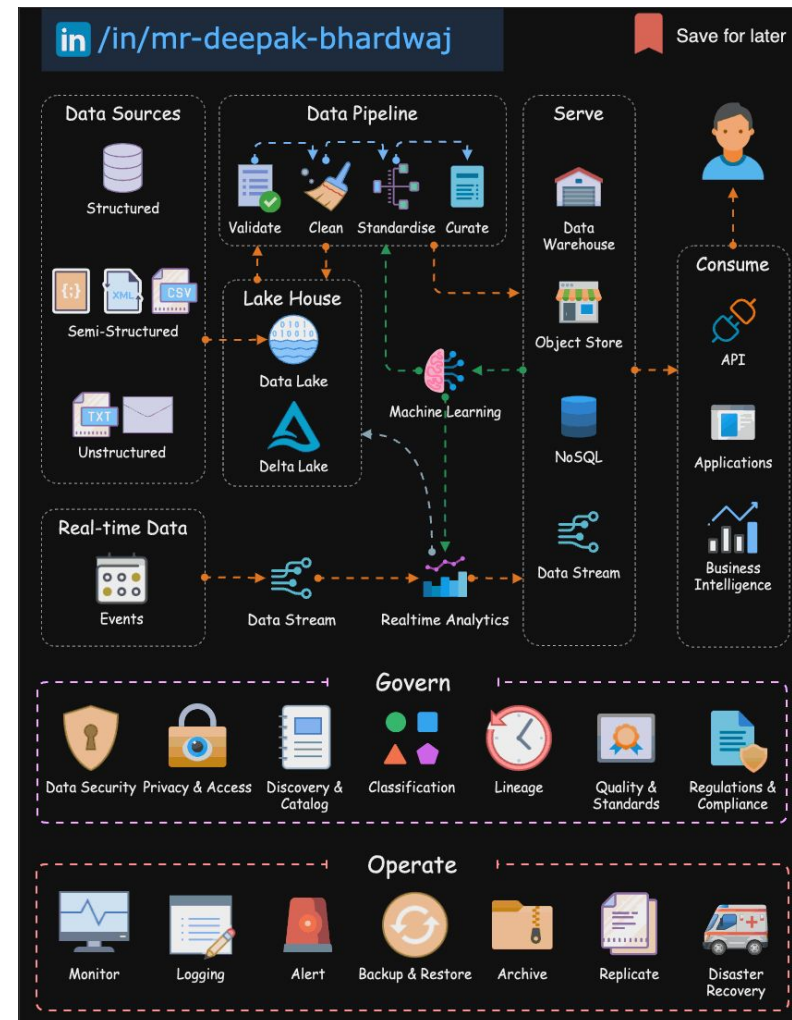
Introduction to Data Architecture

Definition 1

Design of systems to support the evolving **data needs** of an enterprise, achieved by *flexible* and *reversible decisions* reached through a careful evaluation of trade-offs.

Definition 2

Identifying the **data needs** of the enterprise (regardless of structure) and designing and maintaining the *master blueprints* to meet those needs. Using *master blueprints* to guide data integration, control data assets, and align data investments with business strategy.



Architecture Concepts

Domains

A domain is the **real-world subject area** for which you're architecting a system. It represents a business area or an area of expertise (e.g., Sales, Inventory, Finance).

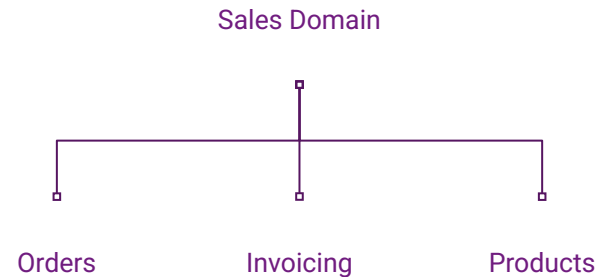
Services

A service is **a set of functionality** whose goal is to accomplish a task.

Relationship

A domain can contain multiple services.

When thinking about what constitutes a domain, the advice is to focus on what the domain represents in the **real world** and work backward.



Data Architecture Concepts

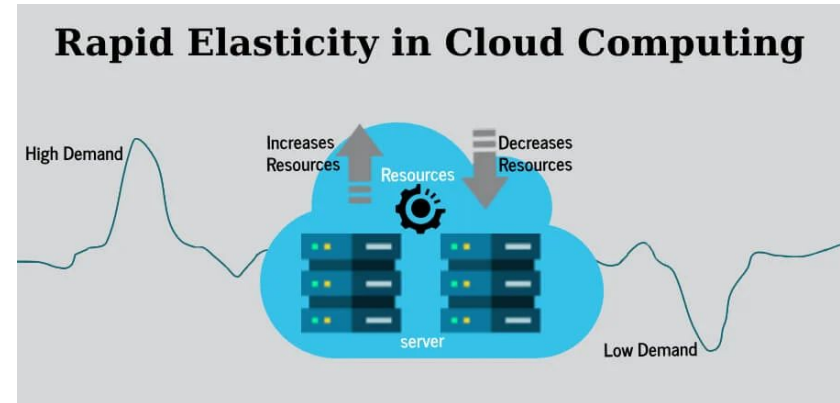
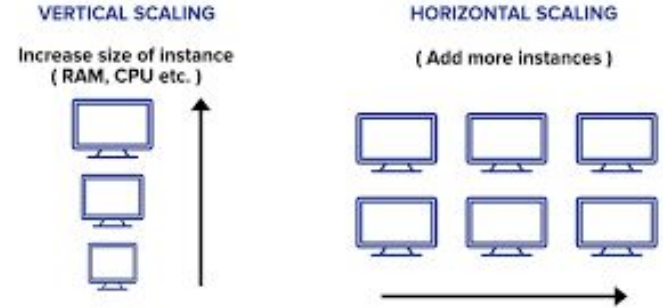
Scalability

Allows us to *increase the capacity* of a system to improve performance and handle the demand. For example, we might want to scale a system to handle a high rate of queries or process a huge data set.

Elasticity

The ability of a scalable system to *scale dynamically*; a highly elastic system can automatically scale up and down based on the current workload.

Scaling up is critical as demand increases, while scaling down saves money in a cloud environment. Modern systems sometimes scale to zero, meaning they can automatically shut down when idle.



<https://www.educba.com/rapid-elasticity-in-cloud-computing/>

Data Architecture Concepts

Availability

The percentage of time an IT service or component is in an operable state.

Reliability

The **system should continue to work correctly** (performing the correct function at the desired level of performance, also known as ***fault-tolerant*** or *resilient*) even when facing adversity (hardware/software *faults*, or human error).

Maintainability

This concept addresses the **long-term work** on the system. Over time, many different people will work on the system (maintaining current behavior and adapting the system to new use cases), and they should all be able to **work on it productively**.

Distributed Systems

Distributed systems enhance scalability, availability, and reliability via **horizontal scaling**, relying on data **replication** and **partitioning(sharding)**.

Node structures include:

- **Leader-follower (master-slave):** Single leader distributes reads/writes to followers. Clients read from leader/followers, write to leader. (Postgres/DuckDB)
- **Multi-leader:** Multiple leaders accept writes, forwarding changes to other nodes. (Certain Cassandra)
- **Leaderless:** Any node can accept writes from clients. (Amazon DynamoDB/Riak)

Prevalent in modern data technologies like NoSQL databases, cloud data warehouses, and object storage systems.

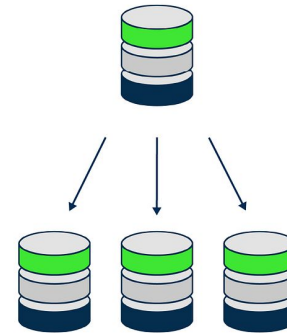
Distributed Systems

Replication

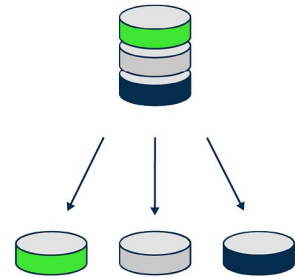
Creates multiple *copies* of the same data and distributing those copies across different servers or nodes. Each copy is referred to as a replica. The primary goal of replication is to enhance **data availability**, **fault tolerance** (*reliability*), and read *scalability*.

Partitioning (Sharding)

Partitions a database into smaller, more manageable subsets called *shards*. Each shard contains a portion of the data. The goal of sharding is to distribute the data and workload across multiple servers, improving **write scalability** and **large datasets management**.



Replication



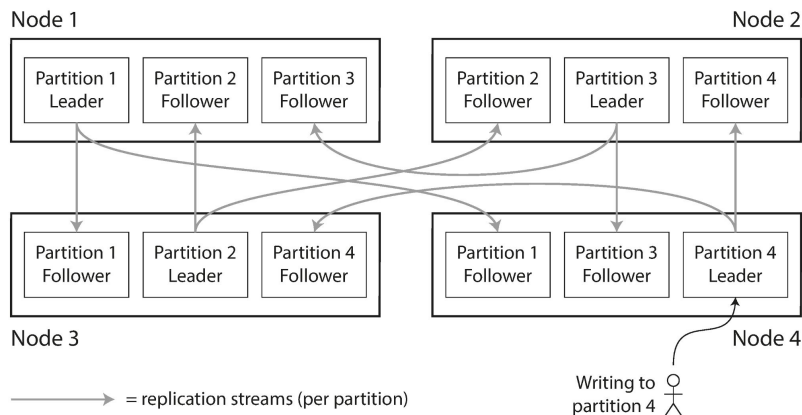
Sharding

Distributed Systems

Partitioning is usually **combined** with replication

- Copies of each partition are stored on multiple nodes.
 - Each record belongs to exactly one partition but may be stored on several nodes for **fault tolerance**.
- A node may store more than one partition.

If a **leader-follower** model is used, each partition's leader is assigned to one node, and its followers are assigned to other nodes. Each node may be the leader for some partitions and a follower for other partitions.



CAP Theorem

The CAP theorem states that a distributed database can only guarantee two out of the three properties:

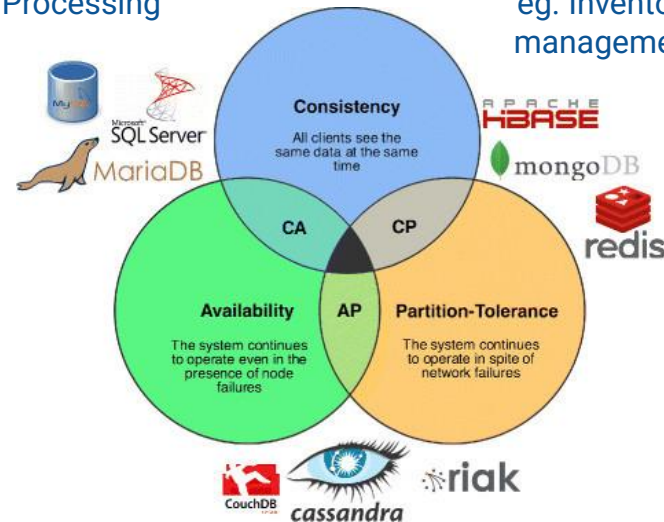
Consistency - All nodes see the same data at the same time, ensuring coherent and accurate reads/writes across the system.

Availability - Every request receives a non-error response, with the system remaining operational despite failures or network partitions.

Partition tolerance - The system continues functioning when network partitions occur, with some nodes unable to communicate with others due to failures or delays.

Choose when network partitions are rare, eg. Financial Transaction Processing

Prioritize data consistency over availability, eg. Inventory management



Favor availability and partition tolerance, eg. Recommendation system

Real World CAP Decisions



Stock Exchanges (CP)

Consistency over Availability. If a network partition occurs, NYSE or SGX halt trading instantly. They lock the doors (sacrificing availability) to prevent disastrous, illegal "double-spending".



Amazon (AP)

Availability over Consistency. Amazon always lets you add items to your cart to save the sale. They resolve inventory issues later (which is why you sometimes get an "out of stock" email later).



Facebook (AP)

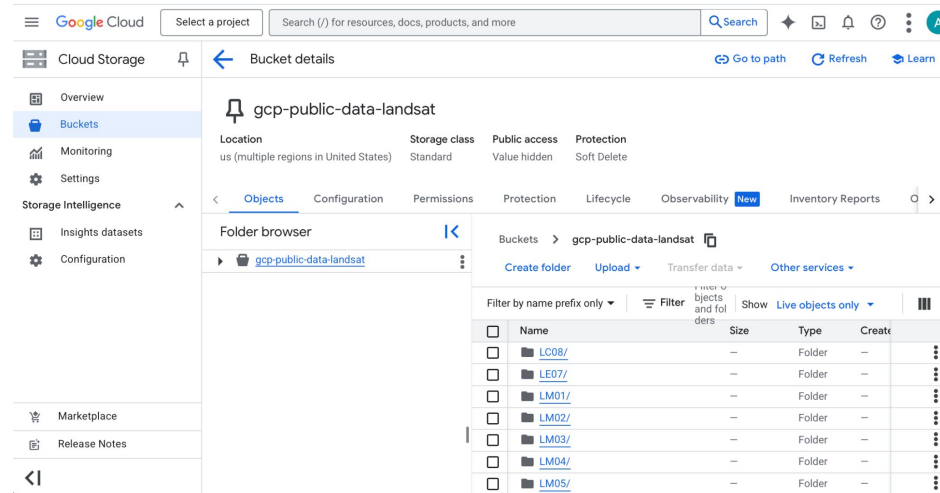
Availability over Consistency. If a server fails, the app still loads. You might see an older post or a delayed "like" count, but keeping the platform running is more important than perfect accuracy.

Types of Data Architecture

Category	Architecture	Tool Examples
Data at Rest	Object Storage	Amazon S3 (AWS), Google Cloud Storage
	Data Lake	AWS Lake Formation (built on Amazon S3), Cloudera
	Data Warehouse	Amazon Redshift, Google BigQuery
	Data Lakehouse	Databricks Lakehouse Platform, Apache Iceberg
Data in Motion	Data Pipelines & Orchestration	Apache Airflow, AWS Glue
	Dataflow Model	Apache Beam, Google Cloud Dataflow
	Lambda Architecture	Apache Spark Streaming, Apache Kafka
	Kappa Architecture	Apache Kafka, Apache Flink
Data Mesh	Data Mesh Platform	Amazon DataZone, Google Dataplex, Azure Purview

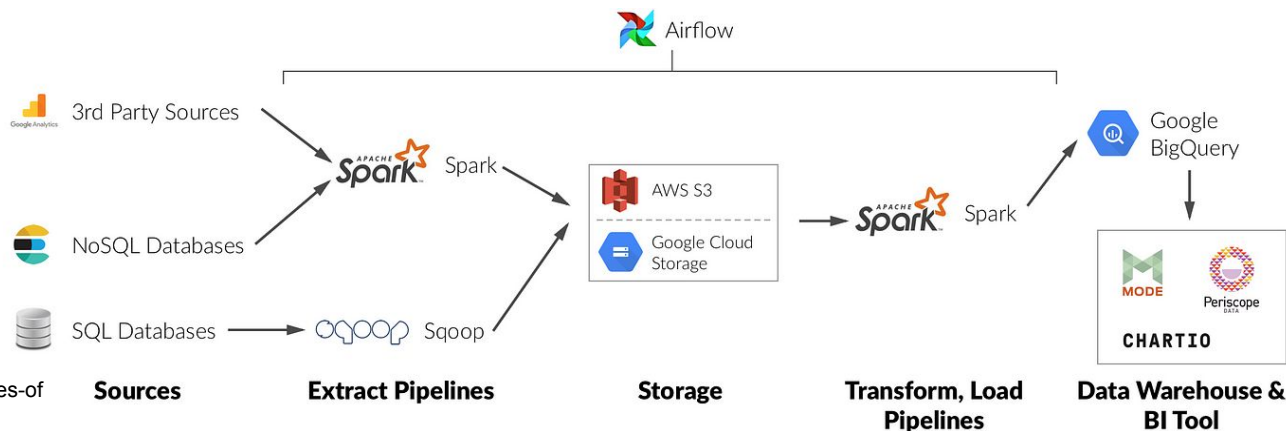
Object Storage

- Data architecture for large unstructured data, stored as distinct units with metadata and unique identifiers.
- Well-suited for vast unstructured data like images, videos, documents, backups. Designed for horizontal scalability by adding nodes.
- Shortcoming: Limited read/write performance for smaller files or high-throughput workloads. Limited advanced search, indexing capabilities compared to file systems/databases. Not ideal for in-place processing or complex analytics.
- Examples: AWS S3, Azure Blob Storage and Google Cloud Storage (GCS).
- [Demo](#)



Data Pipelines

- Data pipelines are the paths data takes through the architecture once it's been created.
- The pipeline includes data collection, storage, access, cleaning, analysis, and presentation.
- It is often thought of as plumbing, where computational and storage resources (pipes) are sized appropriately for efficient data transmission.
- Building a pipeline requires continued maintenance and optimization as part of data infrastructure management.



<https://medium.com/@natekupp/getting-started-the-3-stages-of-data-infrastructure-556dac82e825>

From Data Warehouse To Lakehouse

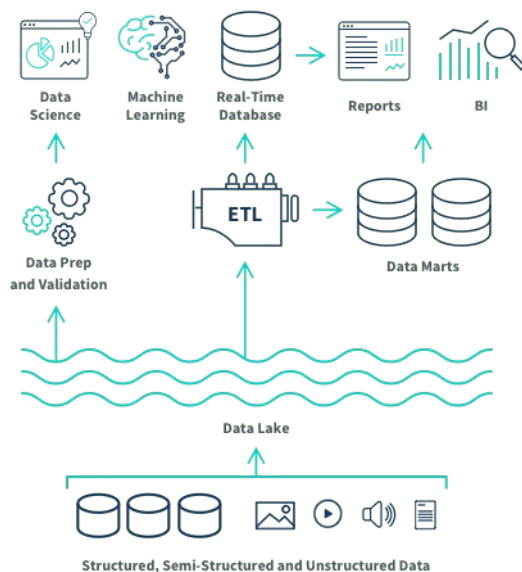
LATE 1980'S

Data Warehouse



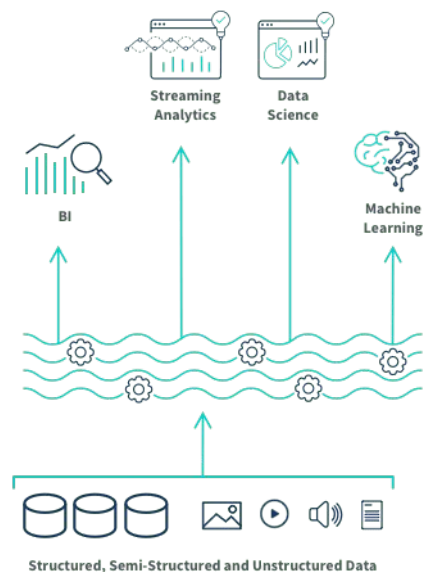
2011

Data Lake



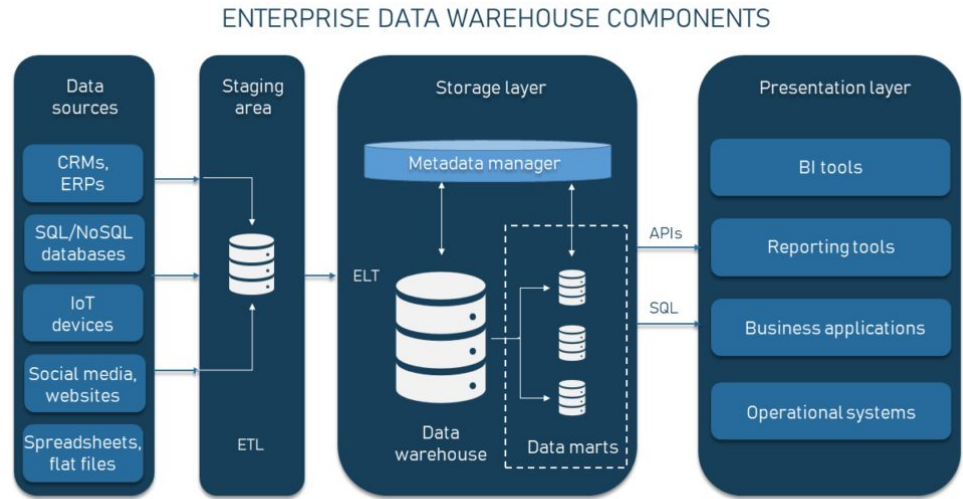
2020

Lakehouse



Data Warehouse

- *Centralized* repository for integrated, historical data from multiple sources.
- Designed for complex queries, reporting, and analysis for business intelligence.
- Data is *extracted, transformed, and loaded* (ETL) to create a unified view.
- Optimized for read-heavy workloads using indexing, partitioning, etc.
- Data is often *denormalized* to reduce complex joins.
- Shortcoming: Complex, time-consuming, and costly to build and maintain.
- Examples: Snowflake, Amazon Redshift, Google BigQuery.



<https://www.altexsoft.com/blog/enterprise-data-warehouse-concepts/>

Data Lake

- Centralized repository for vast raw, unprocessed data in native formats - structured, semi-structured, unstructured (text, images, videos, sensor data, etc.).
- Unlike data warehouses, no predefined schema imposed. Data stored in various raw formats, well-suited for data exploration and experimentation.
- Shortcoming: Became a "data swamp" due to lack of schema management, data cataloging, and discovery tools leading to unmanageable data growth.
- One of the first and most prominent implementation is the Hadoop Distributed File System (HDFS). Data Lake can also be implemented on top of Object Storage such as AWS S3 and GCS.

Data Lakehouse

- Conceived in response to shortcomings of data lakes.
- Incorporates data warehouse controls, management, and structures while housing data like a lake, supporting various query/transformation engines.
- Supports ACID transactions.
- Convergence with cloud data warehouses, which incorporate separate compute/storage, scalability, diverse data type support, and advanced processing integration.
- Examples: Databricks' data lakehouse.

Data Warehouse

The Highly Organized Supermarket

Every item has a barcode, is placed in a specific aisle, and is thoroughly checked before it goes on the shelf.

Highly structured, clean data used for reporting. When the CFO needs to run the end-of-quarter financial report, they need absolute accuracy. The data must be structured, historical, and reliable.



Data Lake

The Raw Data Warehouse

Tesla cars upload petabytes of raw, completely unstructured data every day: video clips of stop signs, sensor readings of rain, and steering wheel angles. You cannot put a video file neatly into a traditional spreadsheet or Data Warehouse. Instead, Tesla dumps all of this raw footage into a massive, cheap Data Lake. They don't know exactly what they are looking for yet, but the data is sitting there waiting for their AI engineers to dive in and use it to train the autopilot.

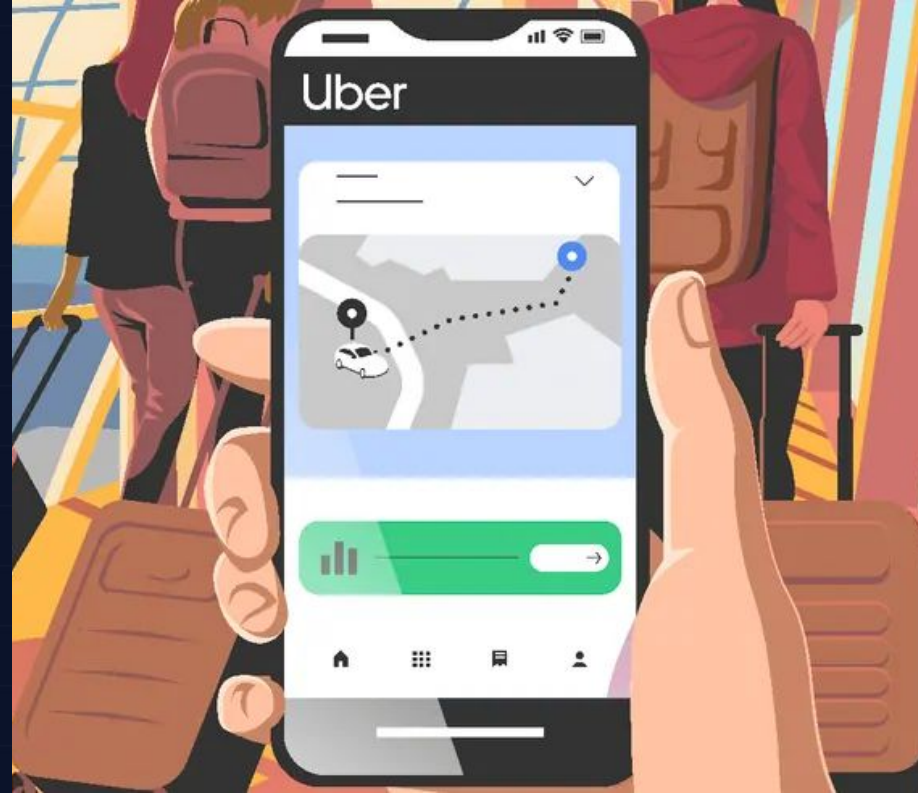
Perfect for when you don't know exactly how you'll use the data yet, but want to save it for future AI analysis or experimentation.



Data Lakehouse

The Best of Both Worlds

Previously, Uber would have had to dump the GPS pings into a cheap Data Lake, and then spend hours copying and formatting the important bits into a Data Warehouse to run payroll. With a Data Lakehouse, they get the best of both worlds. They can cheaply store the ocean of raw GPS pings (the Lake), but apply an organized "layer" on top of it so the finance team can immediately run strict, structured reports for driver payments (the Warehouse) without ever moving the data.



Exercise

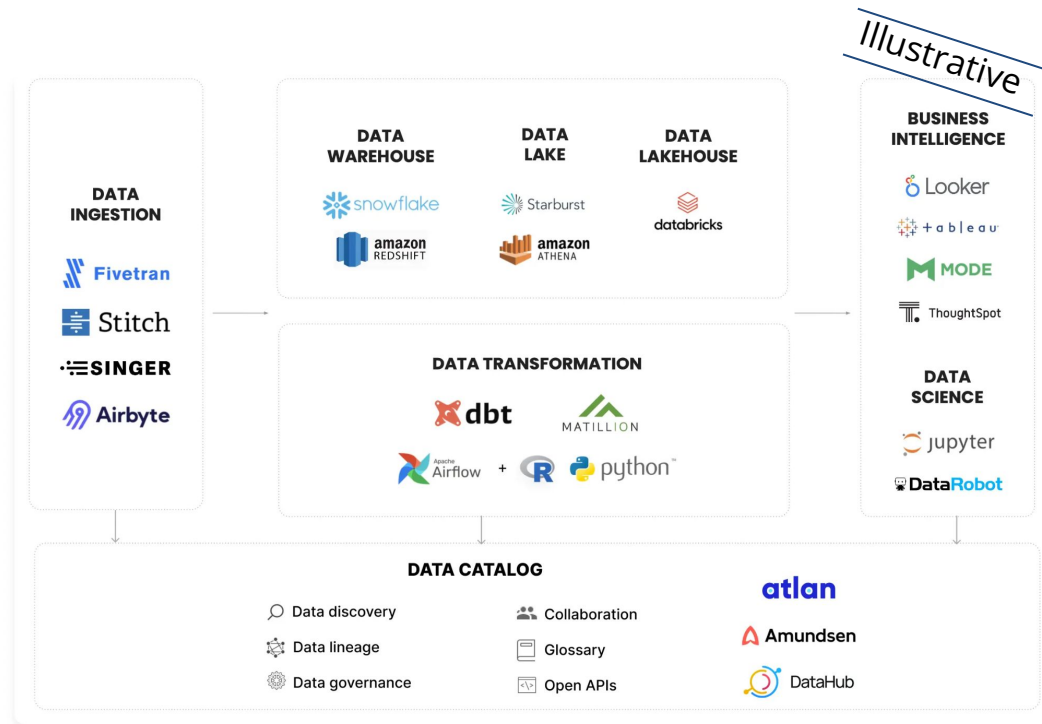
Business Requirement

1. TikTok generates petabytes of these raw, unstructured video files every single day.
2. TikTok's finance team needs to know exactly how many times an ad was clicked so they can bill advertisers precisely.
3. They also need to track exact daily active user counts.
4. TikTok's AI recommendation algorithm. To make it work, the AI needs to look at both things at the same time: it needs to analyze the raw video (e.g., "this video has a cat in it") AND the user data (e.g., "User A liked 5 cat videos today").



Modern Data Stack

- A popular analytics architecture.
- **Modular and Cloud-based:** plug-and-play components for cost-effectiveness.
- **Key Components:** data pipelines, storage, transformation, data management/governance, monitoring, visualization, exploration.
- **Benefits:** modular design reduces complexity, allowing for self-service analytics and pipelines, agile data management, open source or clearly priced tools.



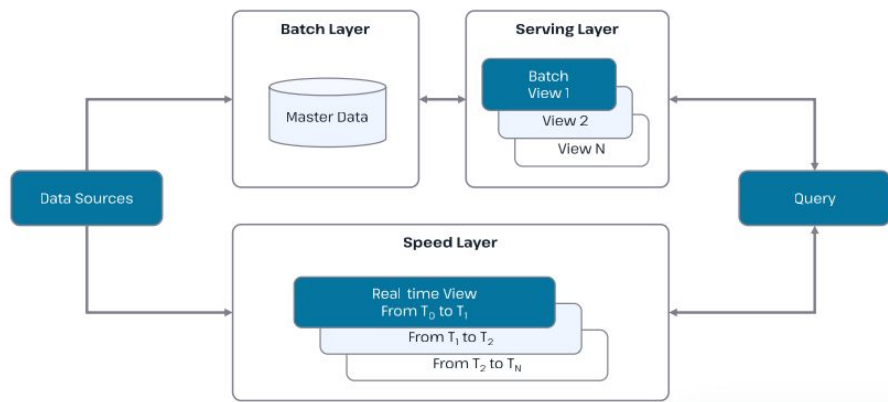
<https://atlan.com/modern-data-stack-101/>

Lambda Architecture



- **Purpose:** It was created to reconcile the need for both highly accurate, historical batch processing and immediate, low-latency stream processing.
- **Speed layer** focuses on low-latency processing, often using NoSQL.
- **Batch layer** processes data in systems like data warehouses, creating precomputed/aggregated views.
- **Serving layer** aggregates query results from both layers for combined view.

- **Shortcoming:** the complexity involved in managing two separate systems, codebases, and data streams



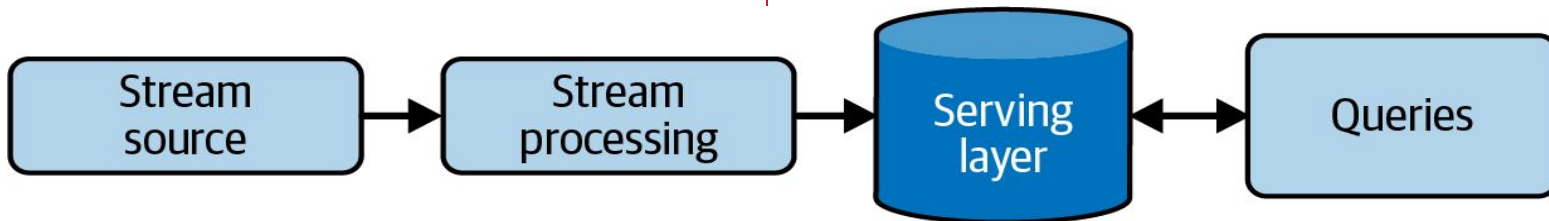
<https://hazelcast.com/foundations/software-architecture/lambda-architecture/>

Kappa Architecture



Uber

- Proposed as an alternative to Lambda architecture.
- **Core Idea:** it suggests using a single **stream-processing platform** for all data tasks - ingestion, storage, serving. Enables a true event-based architecture.
- **Unified Processing:** both real-time and batch processing applied to **same data** by reading live event stream and replaying chunks.
- **Advantage:** simplified architecture
- **Shortcoming:** complicated and expensive in practice. Batch storage/processing remains more efficient and cost-effective for enormous historical datasets.

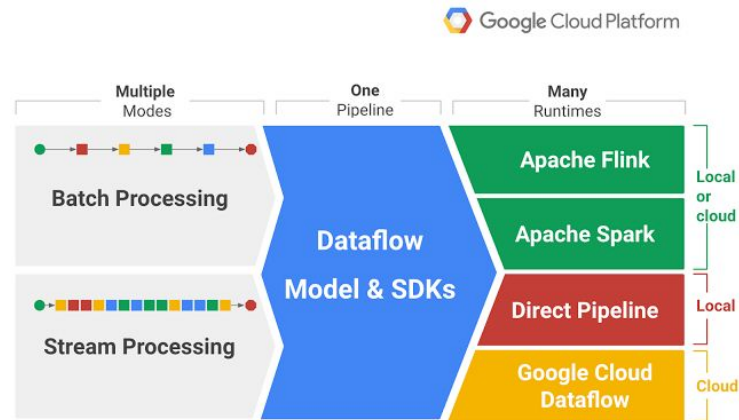


Dataflow Model

- **The Problem:** Unifying batch and stream processing code paths is a central challenge. Kappa architecture uses unified queuing/storage, but still requires different tools for real-time stats and batch aggregation.
- Google's **Dataflow model** and open-source **Apache Beam** framework address this.
- **Core Concept:** In this model, all data is treated as events aggregated over various time windows.
 - **Real-time streams** are considered unbounded event streams.
 - **Data batches** are considered bounded event streams (a special case of streaming).
- **The Philosophy:** This approach allows real-time and batch processing to occur within the same system using similar code, following the principle of "batch as a special case of streaming."
- Frameworks like Flink and Spark embraced similar approach.

POPULAR DATAFLOW PRODUCTS

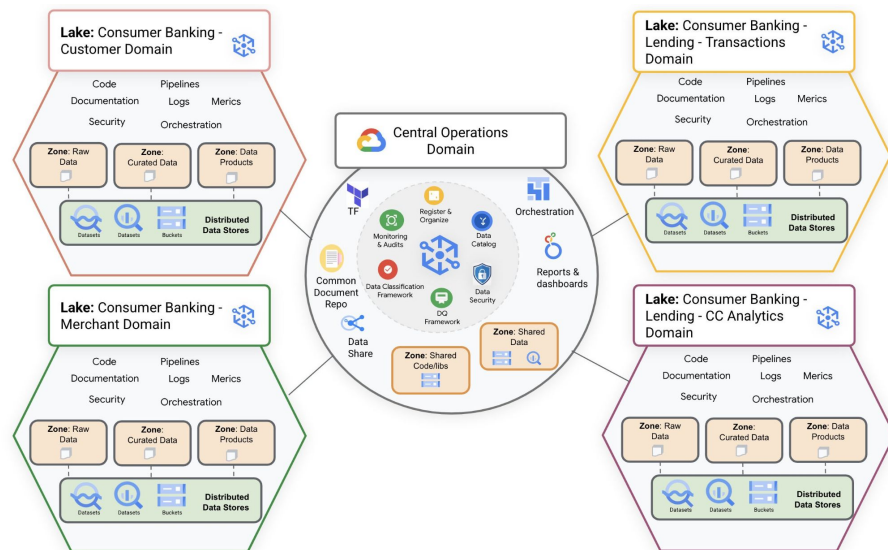
Product	Engine
Google Cloud Dataflow	Apache Beam
Apache Spark (Databricks)	Apache Spark
AWS Glue	Apache Spark
Azure Data Factory	Apache Spark



<https://www.bigdatawire.com/2016/05/02/dataflow-tops-spark-benchmark-test/>

Data Mesh

- A recent response to centralized data platforms like data lakes and warehouses, addressing their limitations.
- **Decentralization is Core:** It is a decentralized approach that applies **domain-driven design** concepts. Instead of data flowing into a centralized platform, different organizational domains (e.g., Sales, Marketing, Inventory) host and serve their own datasets. This reverses the traditional data flow.
- **Tackles Operational vs. Analytical Data:** It addresses the split between operational data (used for daily business) and analytical data (used for insights).
- Four key components of the data mesh:
 - Domain-oriented decentralized data ownership and architecture
 - Data treated as a product
 - Self-serve data infrastructure functioning as a platform
 - Federated computational governance



<https://www.infoworld.com/article/2338279/preview-google-cloud-dataplex-wows.html>

Data Mesh - Google DataPlex

Dataplex overview

Intelligent data management and governance across distributed data

Unified metadata across distributed data

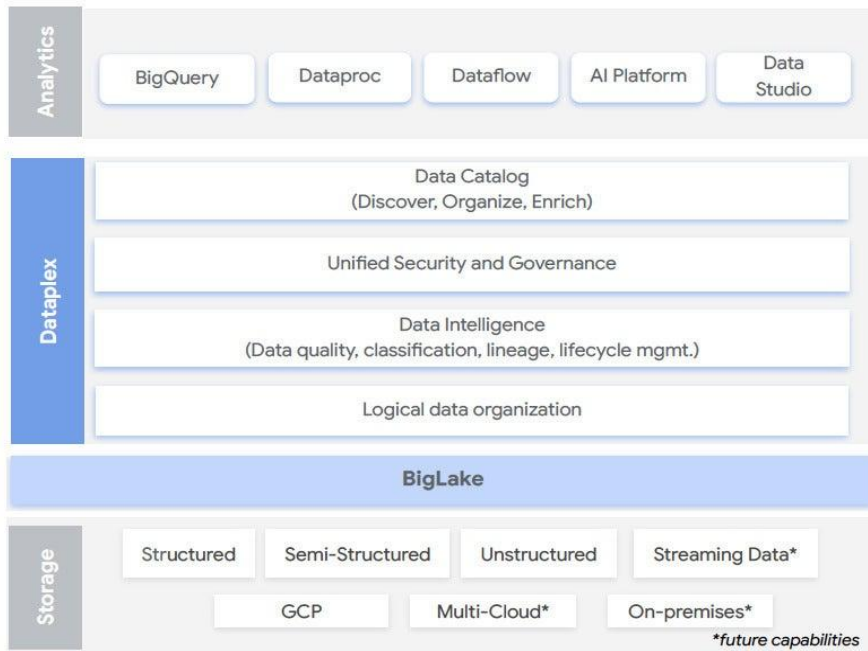
Automatic data discovery & metadata harvesting, enriched with business context. Logically unify and organize your data without any data movement.

Centralized Security & Governance

Central policy management, monitoring and auditing for data authorization, retention, and classification.

Intelligent Data Management

Built-in AI-driven intelligence with data classification, data quality, data lineage and lifecycle management.



<https://www.infoworld.com/article/2338279/preview-google-cloud-dataplex-wows.html>

Principles of Good Data Architecture

01. Data as a Shared Asset

Focus: Breaking down silos, enabling holistic company views

Benefits: Competitive advantage, 360-degree customer insights, cross-functional data correlation

02. User Interfaces for Data Consumption

Focus: Accessible interfaces (OLAP, SQL, APIs), data warehouses/lakes/marts

Benefits: Empowered users, scalable architecture, effective use of familiar tools

03. Security and Access Controls

Focus: Enforcing policies, secure self-service, modern cloud platforms

Benefits: Data protection, compliance, safe real-time data access

04. Common Vocabulary

Focus: Standardizing definitions (catalogs, calendars, KPIs), semantic layers

Benefits: Faster analysis, reduced disputes, consistent understanding

05. Data Curation

Focus: Modeling, cleansing, curating dimensions/measures

Benefits: Improved data quality, better user experience, maximized data value

06. Minimizing Data Copies and Movement

Focus: Reducing unnecessary movement, leveraging cloud scalability

Benefits: Lower costs, increased data freshness, greater agility

Modern Principles

01. Data as Product Focus: Quality, discoverability, documentation & usability Benefits: Higher data quality, better collaboration	02. Domain-Oriented Decentralized Ownership Focus: Align data ownership with domain expertise Benefits: Scalability, domain-specific speed	03. Cloud-Native & Infrastructure as Code Focus: Elasticity, automation, repeatable setups Benefits: Fast provisioning, automation, traceability	04. Event-Driven Architecture Focus: Real-time data flows, system responsiveness Benefits: Real-time insights, decoupled systems
05. Interoperability & Open Standards Focus: System integration, vendor independence Benefits: Vendor flexibility, smooth integration	06. Metadata-Driven Architecture Focus: Automation, data lineage, discovery Benefits: Enhanced observability, automation	07. Resilience & Fault Tolerance Focus: Service availability, data protection Benefits: High availability, user satisfaction	08. Observability & Monitoring Focus: Bottleneck identification, operational optimization Benefits: Proactive issue resolution, performance optimization

Join at
slido.com
#DS6202

